Module 8 – Azure Integration

Identity Provider

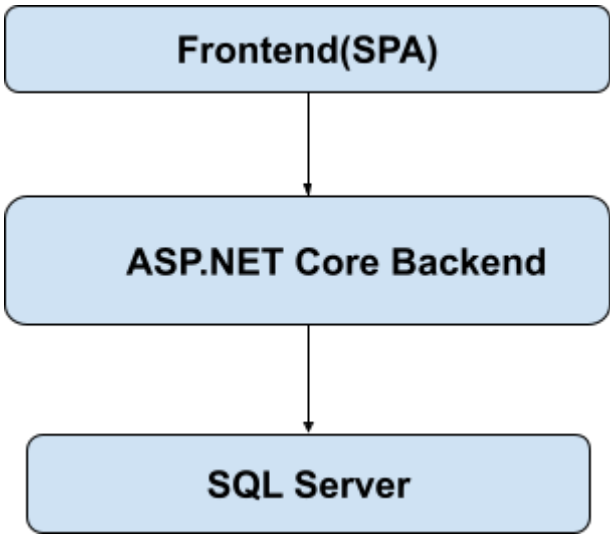
The system integrates Azure Active Directory (Microsoft Entra ID) as the external Identity Provider (IdP).

Azure AD manages:

- User authentication
- Token issuance
- Role claims
- Identity lifecycle

The application does not store user passwords locally.

Before Azure:-



But there is **no centralized identity management** yet.

Anyone who hits the API could potentially access endpoints unless you implement manual auth.

That’s not enterprise-ready.

Why Azure Active Directory (Now called Microsoft Entra ID)

We integrate:

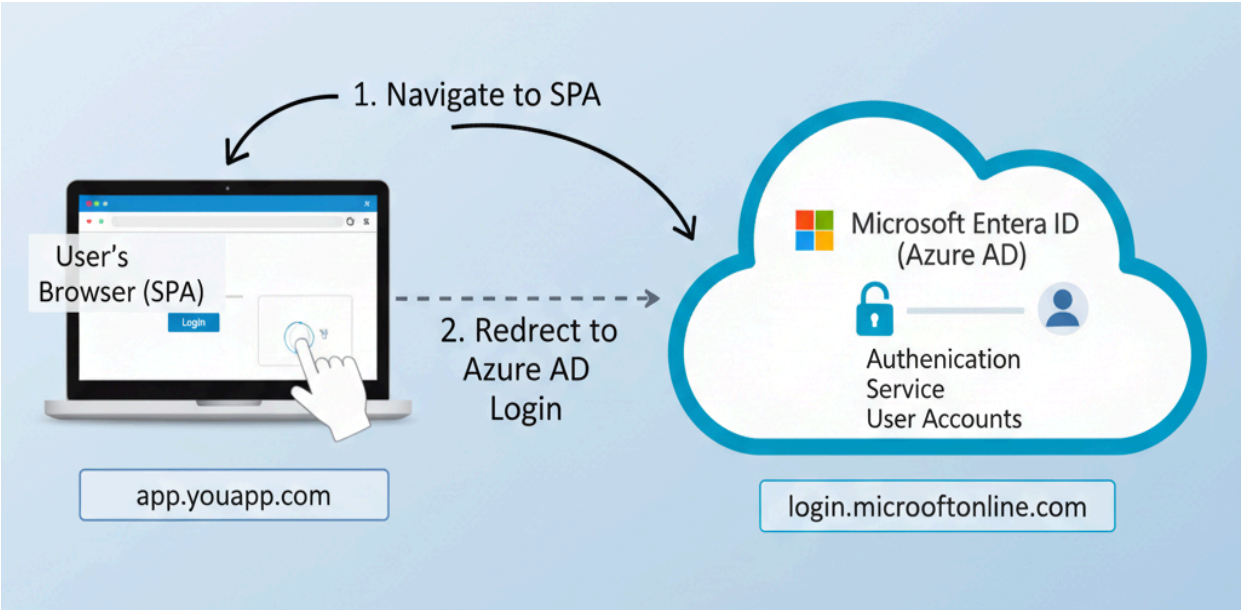
Microsoft Entra ID (Azure AD)

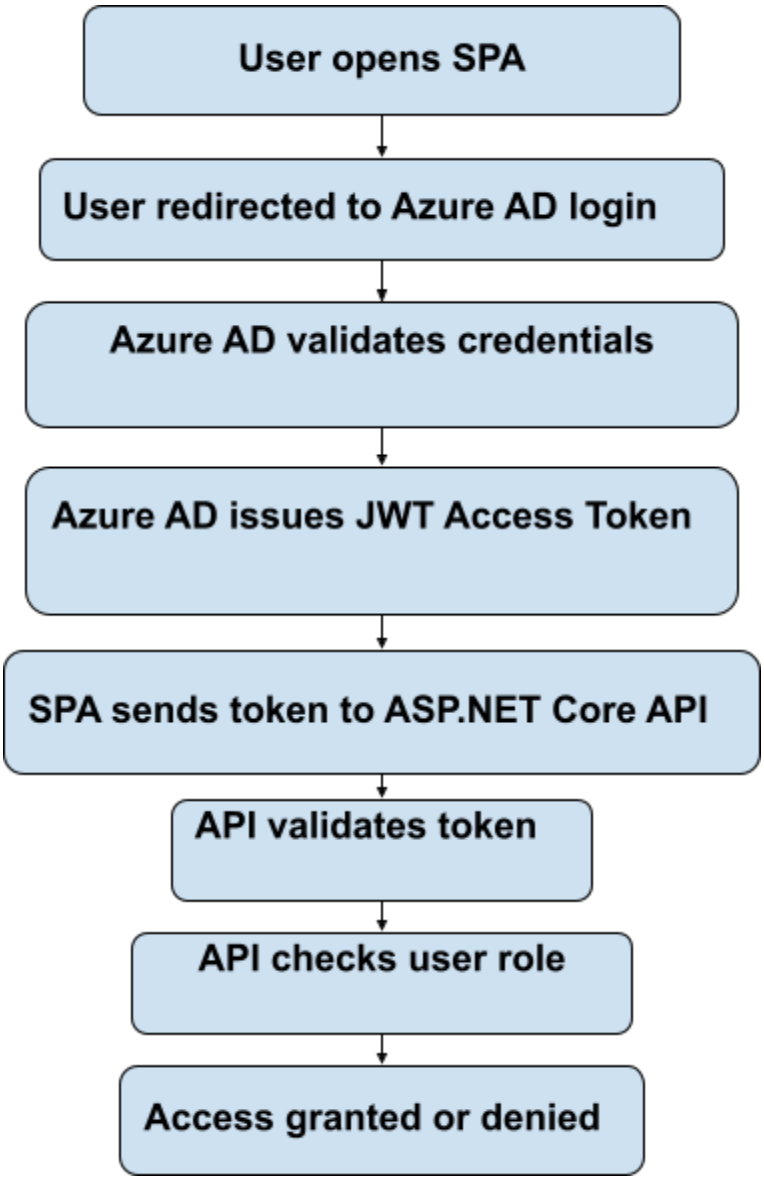
✔ Why?

Because it provides:

Feature	Why Important
Centralized identity	One identity system for all users
Role-based access control	Doctors ≠ Receptionists ≠ Admins
Token-based security (JWT)	Secure API communication
Enterprise compliance	Healthcare requires security
MFA support	Extra protection for admins
Easy cloud scalability	Ready for Azure deployment later

Authentication Flow





Azure steps:-

Step 1: User Login

- SPA redirects to Azure AD
- Azure AD authenticates

Step 2: Token Issued

Azure AD returns:

- Access Token (JWT)
- Contains:
    - User ID
    - Email
    - Role (Doctor/Admin/etc.)
    - Expiry

Step 3: API Validation

ASP.NET Core:

- Uses Microsoft.Identity.Web
- Validates:
    - Signature
    - Issuer
    - Audience
    - Expiry

Token-Based Security Model

- Authentication mechanism: OAuth 2.0 / OpenID Connect
- Token type: JWT (JSON Web Token)
- Token contains:
    - User ID

    - Email
    - Assigned role
    - Expiration time
    - Issuer information

The API validates:

- Token signature
- Token issuer
- Audience
- Expiration

Role-Based Access Design

We define Azure AD roles:

Role	Access
Admin	Full access
Doctor	View + update appointments
Receptionist	Create appointments
Nurse	Limited view

Example Authorization in ASP.NET Core

```
[Authorize(Roles = "Admin")]
public IActionResult GetAllPatients()
{
    return Ok();
}

[Authorize(Roles = "Doctor")]
public IActionResult UpdateAppointment()
{
    return Ok();
}
```

This ensures:

- Doctors cannot delete departments
- Receptionists cannot modify medical records
- Admins can manage everything

Why This Is Critical in Healthcare

Healthcare systems require:

- Patient data privacy
- Role segregation
- Audit compliance
- Secure authentication

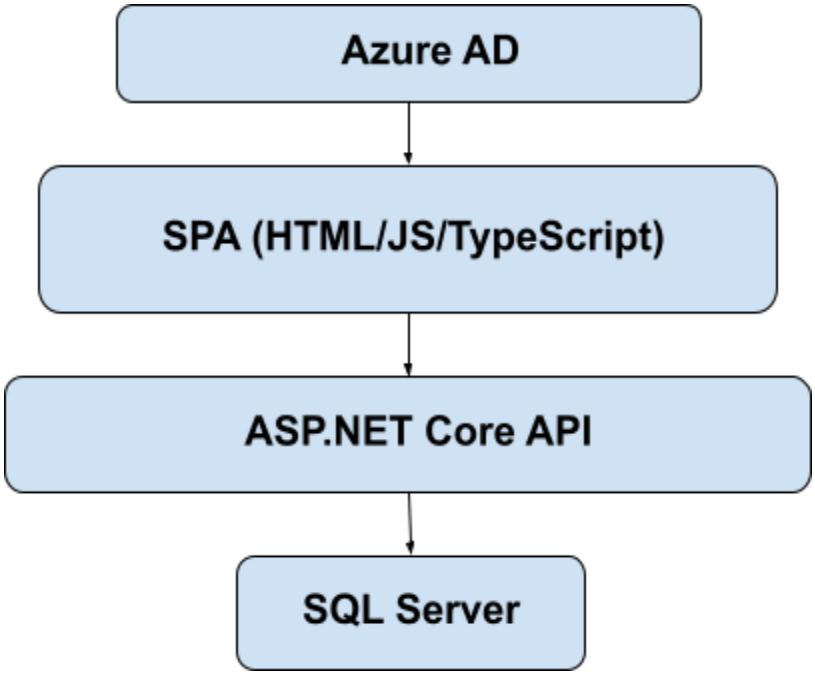
Azure AD ensures:

- Identity governance
- Enterprise-level encryption
- MFA support

- Secure token validation
- No password storage in your app

Your system **never stores passwords**.

Final architecture:



Azure AD becomes the **identity provider (IdP)**.

Our backend becomes a **resource server**.

Frontend Integration Concept

The SPA:

- Uses MSAL.js for authentication
- Redirects users to Azure AD

- Stores access token securely in memory
- Sends bearer token in HTTP headers:

Authorization: Bearer <access_token>

Deployment Independence

This integration remains conceptual:

- No Azure tenant deployment required
- No mandatory App Registration configuration
- No certificate management required
- No live cloud hosting dependency

The design supports future Azure deployment without structural changes.